

Lab#03

Department of Computer Science (7th Semester)

Course: DevOps

Name:

Zahra Bashir

SUBMITTED TO:

Sir Waqas Saleem

Registration Number

23-BCS-092(Section-A)

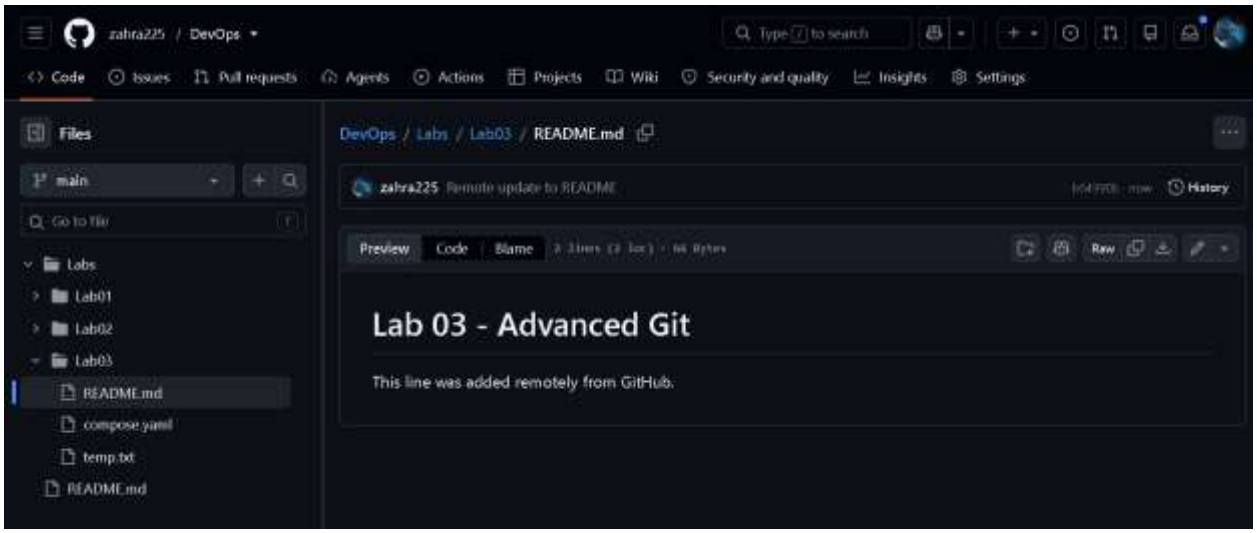
Lab Title: Working with Git History, Stashing, and Reverting Commits

Github link: zahra225

Task 1 – Handling Local and Remote Commit Conflicts (Pull vs Pull --rebase):

1. **Open your repository on GitHub and edit the README.md file directly in the browser.**

- Add a new line such as:
- This line was added remotely from GitHub.
- Commit your change on GitHub (e.g., message: *Remote update to README*).



2. **On your local machine, open the same repository folder.**

- **Edit the same README.md file locally, for example:**
  - This line was added locally.
- **Stage and commit your change:**
  - git add README.md
  - git commit -m "Local update to README"

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git add README.md
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git log --oneline -3
1e4bca0 (HEAD -> main, origin/main, origin/HEAD) Initial Lab03 structure
4f16bb9 Add screenshots to Lab02
b8c29bc git add Labs/Lab02
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git commit -m "Local update to README"
[main cb5c3b2] Local update to README
 1 file changed, 1 insertion(+)
```

3. **Try to push:**

- git push origin main

You'll see an error message similar to:

! [rejected] main -> main (fetch first)

error: failed to push some refs to 'github.com:username/repo.git'

hint: Updates were rejected because the remote contains work that you do not have locally.

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git push origin main
To github.com:zahra225/DevOps.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'github.com:zahra225/DevOps.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

4. **To fix this, pull the latest changes from remote:**

- git pull --no-rebase origin main

Git will **merge** your local and remote commits, creating a *merge commit*.

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git pull --no-rebase origin main
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (5/5), 1.11 KiB | 284.00 KiB/s, done.
From github.com:zahra225/DevOps
 * branch            main       -> FETCH_HEAD
   1e4bca0..b64990b  main       -> origin/main
Auto-merging Labs/Lab03/README.md
CONFLICT (content): Merge conflict in Labs/Lab03/README.md
Automatic merge failed; fix conflicts and then commit the result.
zahra225@ubuntu:~/DevOps/Labs/Lab03$ nano README.md
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git add README.md
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git commit -m "Merge remote and local README changes"
[main 06fb67d] Merge remote and local README changes
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git log --oneline --graph -5
*   06fb67d (HEAD -> main) Merge remote and local README changes
|\
| * b64990b (origin/main, origin/HEAD) Remote update to README
* | cb5c3b2 Local update to README
|/
* 1e4bca0 Initial Lab03 structure
* 4f16bb9 Add screenshots to Lab02
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

5. **Push again:**

- git push -u origin main

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git push -u origin main
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 2 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 967 bytes | 107.00 KiB/s, done.
Total 10 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 1 local object.
To github.com:zahra225/DevOps.git
   b64990b..06fb67d  main -> main
branch 'main' set up to track 'origin/main'.
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

6. **Now repeat the same process, but this time use rebase instead of merge:**

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ echo "Second local line." >> README.md
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git add README.md
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git commit -m "Local update 2"
[main 6908621] Local update 2
 1 file changed, 1 insertion(+)
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git pull --rebase origin main
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (5/5), 1.09 KiB | 139.00 KiB/s, done.
From github.com:zahra225/DevOps
 * branch            main       -> FETCH_HEAD
   06fb67d..ab105a6  main       -> origin/main
Auto-merging Labs/Lab03/README.md
CONFLICT (content): Merge conflict in Labs/Lab03/README.md
error: could not apply 6908621... Local update 2
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
hint: Disable this message with "git config set advice.mergeConflict false"
Could not apply 6908621... # Local update 2
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git add README.md
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git rebase --continue
[detached HEAD fa6003f] Local update 2
 1 file changed, 4 insertions(+)
Successfully rebased and updated refs/heads/main.
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git log --oneline --graph -5
* fa6003f (HEAD -> main) Local update 2
* ab105a6 (origin/main, origin/HEAD) Remote update 2
*   06fb67d Merge remote and local README changes
|\
| * b64990b Remote update to README
* | cb5c3b2 Local update to README
|/
```

Push again:

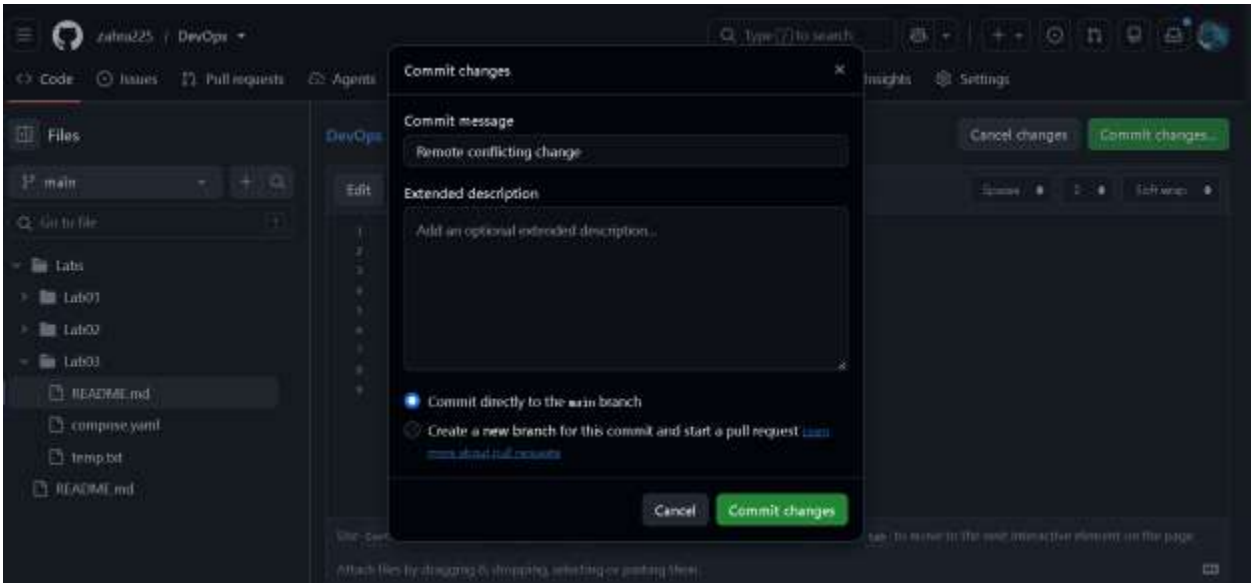
```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git push -u origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 2 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 563 bytes | 281.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:zahra225/DevOps.git
   ab105a6..fa6003f  main -> main
branch 'main' set up to track 'origin/main'.
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

Summary:

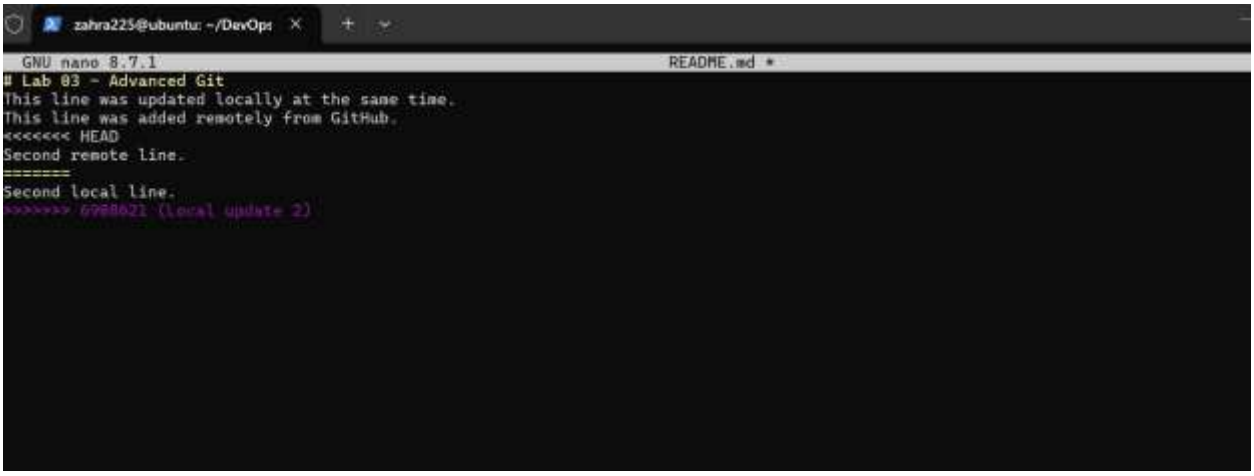
- git pull --no-rebase merges remote changes with your local commits, possibly creating a merge commit.
- git pull --rebase keeps commit history linear by replaying your local commits on top of the updated remote branch.
- Both are ways to resolve push conflicts when your local repo is behind the remote.

Task 2 – Creating and Resolving Merge Conflicts Manually

1. On GitHub (remote), open your README.md file and change an existing line.



2. On your **local machine**, edit the **same line** in the same file but make a **different change**:



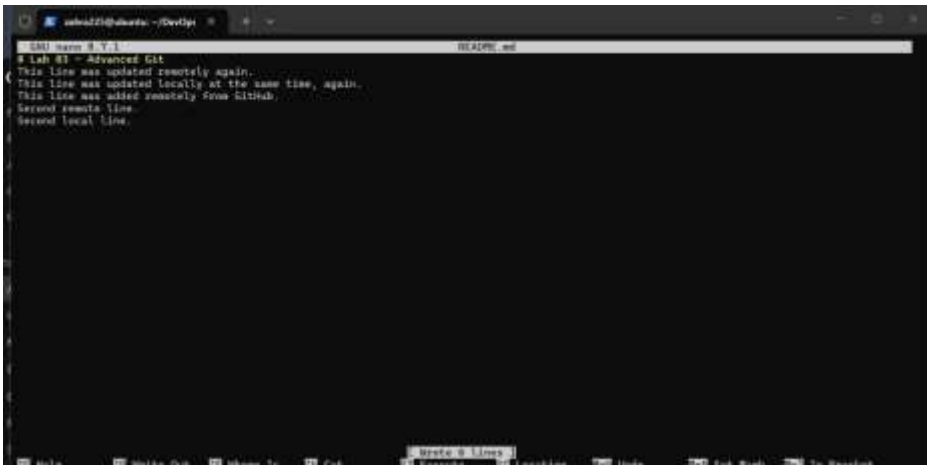
3. Stage and commit your local change:

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git add README.md
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git commit -m "Local conflicting change"
[main e020b00] Local conflicting change
 1 file changed, 1 insertion(+), 1 deletion(-)
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git log --oneline -3
e020b00 (HEAD -> main) Local conflicting change
99110e4 (origin/main, origin/HEAD) Local conflicting change
fa6003f Local update 2
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git push -u origin main
To github.com:zahra225/DevOps.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'github.com:zahra225/DevOps.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

4. Try to push:

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git push -u origin main
To github.com:zahra225/DevOps.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'github.com:zahra225/DevOps.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

5. Pull with rebase to bring in remote changes:
6. Open the README.md file in your editor — you'll see conflict markers like this:



7. After fixing the file, mark the conflict as resolved and continue the rebase:

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git add README.md
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git rebase --continue
[detached HEAD b740233] Local conflicting change
 1 file changed, 1 insertion(+), 3 deletions(-)
Successfully rebased and updated refs/heads/main.
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git log --oneline --graph -5
* b740233 (HEAD -> main) Local conflicting change
* 4ee5cb5 (origin/main, origin/HEAD) Remote conflicting change
* 99110e4 Local conflicting change
* fa6003f Local update 2
* ab105a6 Remote update 2
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

8. Finally, push your changes:

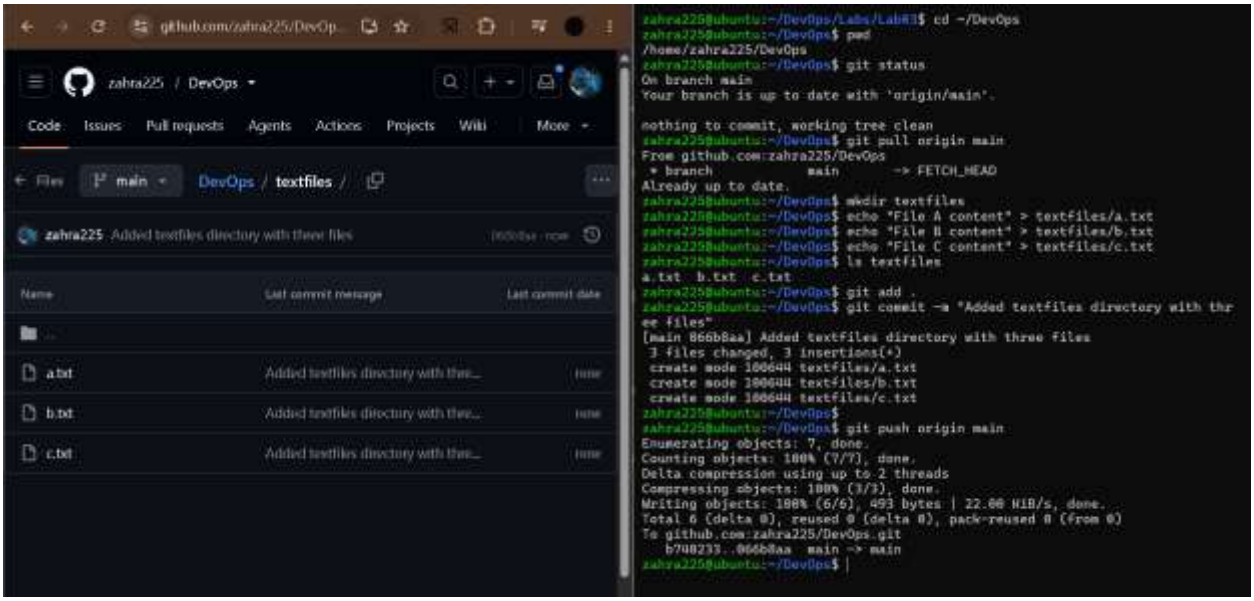
```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git push -u origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 2 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 533 bytes | 133.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To github.com:zahra225/DevOps.git
   4ee5cb5..b740233  main -> main
branch 'main' set up to track 'origin/main'.
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

Summary:

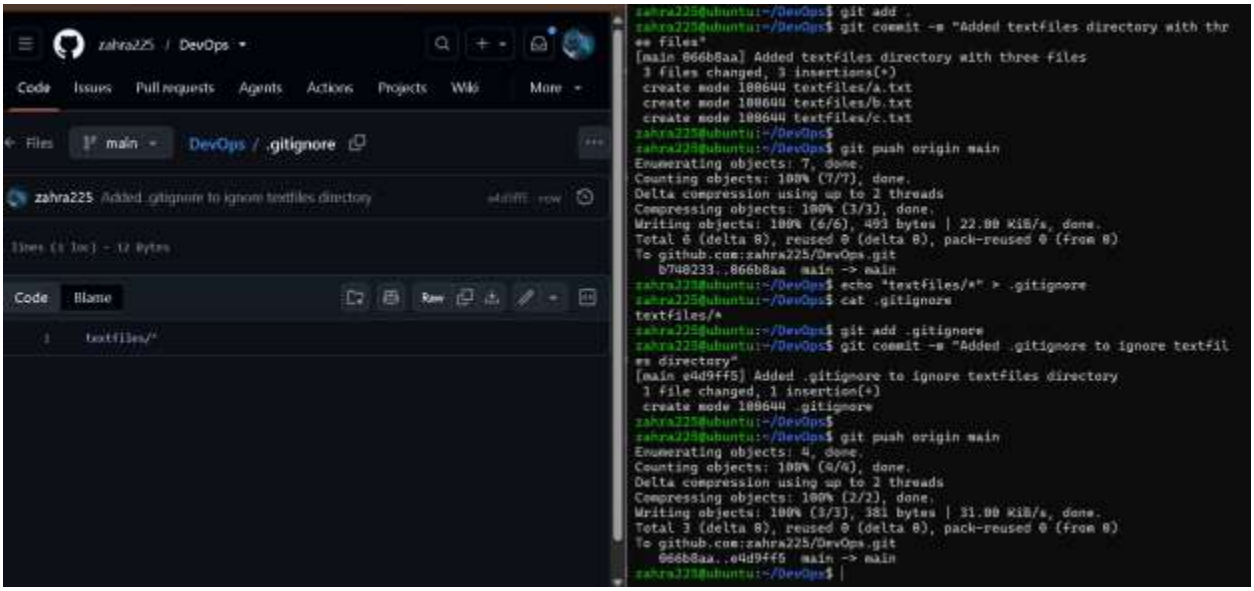
- When two people edit the same line, Git will pause and ask for manual conflict resolution.
- Conflict markers appear in the file and you must choose what to keep.
- After resolving, continue the rebase and push to remote.
- This process ensures both sets of changes are handled safely.

Task 3 – Managing Ignored Files with .gitignore and Removing Tracked Files

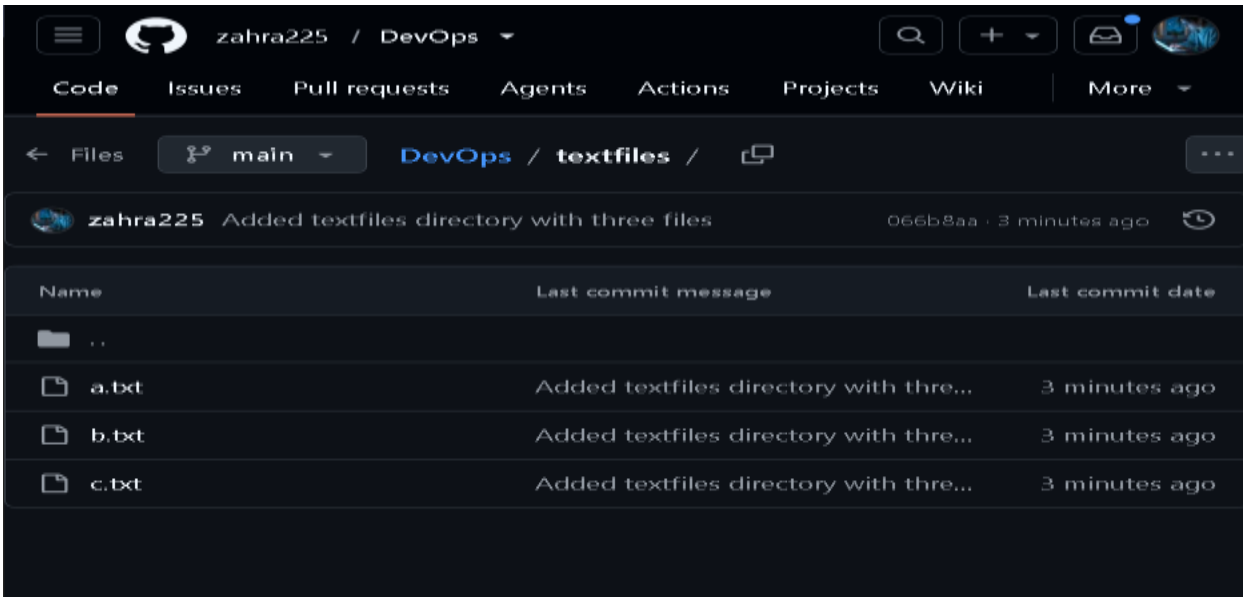
1. Create a new folder named textfiles inside your repository:
2. Inside the textfiles folder, create three text files:
3. Add and commit the new directory:



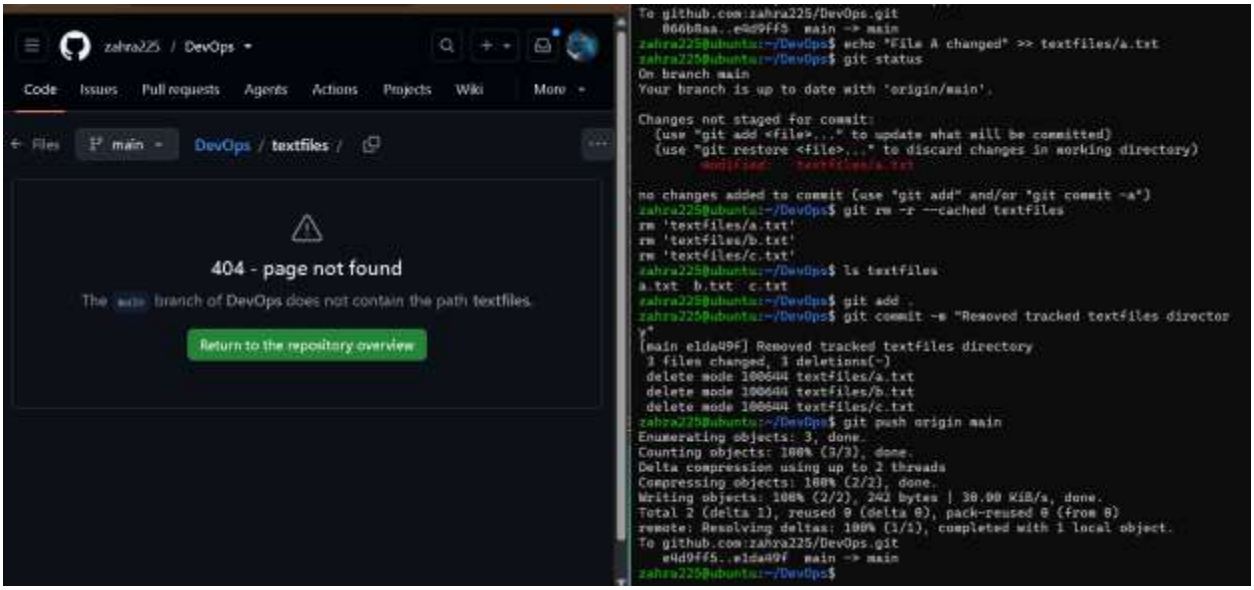
4. Now, create a .gitignore file in the root of your repository:
5. Add and commit the .gitignore file:



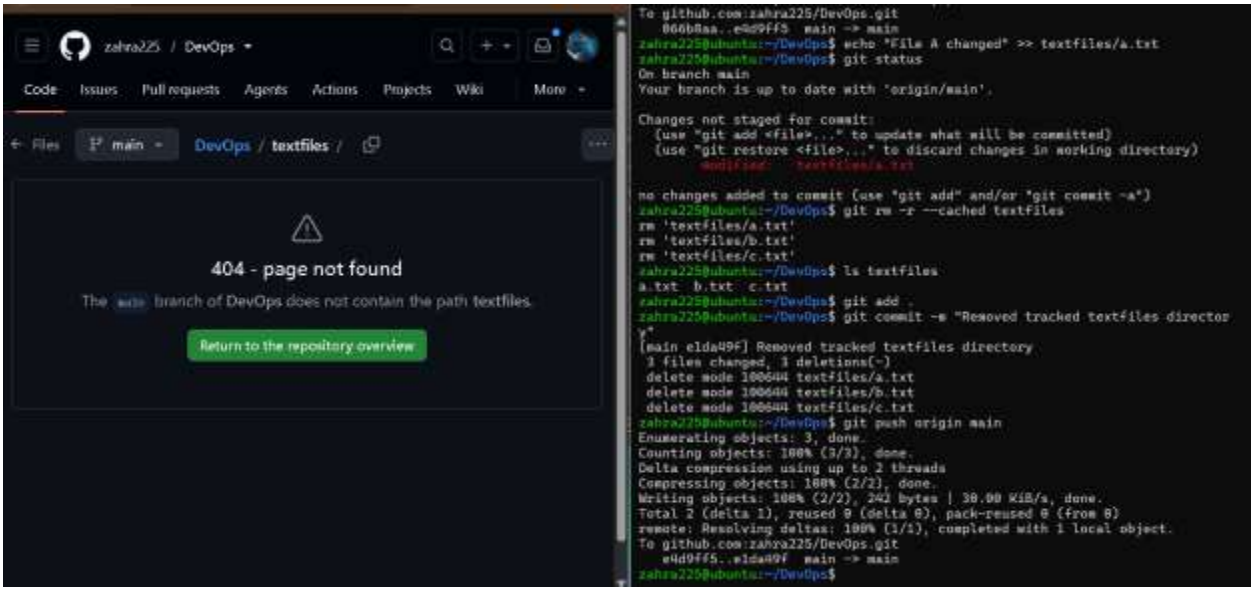
6. Go to **GitHub** and check your repository — notice that the textfiles directory is **still visible** on the remote.



7. To remove already tracked files from Git (but not from your local system), run:
8. Commit and push again:



9. Check your GitHub repository again — the textfiles folder should now be **deleted remotely**.



Summary:

- .gitignore prevents new files from being tracked, but does not remove ones already tracked.
- To untrack files that are already committed, use git rm --cached.
- This lets you keep files locally but remove them from the remote repo.

Task 4 – Create Temporary Changes and Use git stash

1. Create a feature-branch.
2. Modify any file (for example, README.md) by adding a few test lines.

```
zahra225@ubuntu:~/DevOps$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
zahra225@ubuntu:~/DevOps$ cd ~/DevOps
zahra225@ubuntu:~/DevOps$ git checkout main
Already on 'main'
Your branch is up to date with 'origin/main'.
zahra225@ubuntu:~/DevOps$ git pull origin main
From github.com:zahra225/DevOps
 * branch            main       -> FETCH_HEAD
Already up to date.
zahra225@ubuntu:~/DevOps$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
zahra225@ubuntu:~/DevOps$ git checkout -b feature-branch
Switched to a new branch 'feature-branch'
zahra225@ubuntu:~/DevOps$ git branch
* feature-branch
  main
zahra225@ubuntu:~/DevOps$ cd ~/DevOps/Labs/Lab03
zahra225@ubuntu:~/DevOps/Labs/Lab03$ echo "Feature line 1" >> README.md
zahra225@ubuntu:~/DevOps/Labs/Lab03$ echo "Feature line 2" >> README.md
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git add README.md
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git commit -m "Changes the README.md f
ile"
[feature-branch e91c122] Changes the README.md file
 1 file changed, 2 insertions(+)
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git log --oneline -3
e91c122 (HEAD -> feature-branch) Changes the README.md file
e1da49f (origin/main, origin/HEAD, main) Removed tracked textfiles director
y
e4d9ff5 Added .gitignore to ignore textfiles directory
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

3. Modify any file (for example, README.md) by adding a few new lines.
4. Without committing or stashing the changes, try to switch to another branch:

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ sed -i 's/Feature line 2/Feature line
2 - edited, not committed/' README.md
zahra225@ubuntu:~/DevOps/Labs/Lab03$ echo "Feature line 3" >> README.md
zahra225@ubuntu:~/DevOps/Labs/Lab03$ cat README.md
# Lab 03 - Advanced Git
This line was updated remotely again.
This line was updated locally at the same time, again.
This line was added remotely from GitHub.
Second remote line.
Second local line.
Feature line 1
Feature line 2 - edited, not committed
Feature line 3
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git status
On branch feature-branch
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git checkout main
error: Your local changes to the following files would be overwritten by ch
eckout:
        Labs/Lab03/README.md
Please commit your changes or stash them before you switch branches.
Aborting
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

5. To fix this, **stash your changes**:

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git stash
Saved working directory and index state WIP on feature-branch: e91c122 Chan
ges the README.md file
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git stash list
stash@{0}: WIP on feature-branch: e91c122 Changes the README.md file
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

6. Try switching branches again — it should now work:

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

7. Return to your previous branch (for example, feature-branch):

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git checkout feature-branch
Switched to branch 'feature-branch'
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

8. Check your working directory status:

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git status
On branch feature-branch
nothing to commit, working tree clean
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

9. Now restore your stashed changes:

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git stash pop
On branch feature-branch
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (c629ac00b7a0984b6d88f7de8fcd8f8c9a46a6f0)
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

Summary:

- git stash saves your uncommitted changes, letting you switch branches safely.
- You can restore stashed changes anytime with git stash pop.
- This is useful for quickly saving work-in-progress without cluttering commit history.

Task 5 – Checkout a Specific Commit Using git log

1. View commit history:

```
zahra225@ubuntu:~/DevOps$ git log --oneline
e1da49f (HEAD -> main, origin/main, origin/HEAD) Removed tracked textfiles
directory
e4d9ff5 Added .gitignore to ignore textfiles directory
066b8aa Added textfiles directory with three files
b7a0233 Local conflicting change
4ee5cb5 Remote conflicting change
99110e4 Local conflicting change
fa6003f Local update 2
ab105a6 Remote update 2
86fb67d Merge remote and local README changes
cb5c3b2 Local update to README
b04990b Remote update to README
1e4bca0 Initial Lab03 structure
4f16bb9 Add screenshots to Lab02
b8c29bc git add Labs/Lab02
2288c1b "Delete Lab02"
381d9af Add files via upload
fd3ab6d Rename lab2 to Lab02
80b9a9f Add Lab 02
7195127 README.md
c0383eb Add README for DevOps Lab 01
6f0fb1c Add Lab 01 solution
zahra225@ubuntu:~/DevOps$
```

2. Copy any previous commit hash (e.g., a1b2c3d).

3. Checkout that commit:

```
7195127 README.md
c0383eb Add README for DevOps Lab 01
6f8fb1c Add Lab 01 solution
zahra225@ubuntu:~/DevOps$ git checkout e4d9ff5
Note: switching to 'e4d9ff5'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at e4d9ff5 Added .gitignore to ignore textfiles directory
zahra225@ubuntu:~/DevOps$ git log --oneline -3
e4d9ff5 (HEAD) Added .gitignore to ignore textfiles directory
066b8aa Added textfiles directory with three files
b740233 Local conflicting change
zahra225@ubuntu:~/DevOps$ cat Labs/Lab03/README.md
# Lab 03 - Advanced Git
This line was updated remotely again.
This line was updated locally at the same time, again.
This line was added remotely from GitHub.
Second remote line.
Second local line.
zahra225@ubuntu:~/DevOps$ git status
HEAD detached at e4d9ff5
nothing to commit, working tree clean
zahra225@ubuntu:~/DevOps$
```

4. To return to your main branch:

```
zahra225@ubuntu:~/DevOps$ git checkout main
Previous HEAD position was e4d9ff5 Added .gitignore to ignore textfiles directory
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
zahra225@ubuntu:~/DevOps$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
zahra225@ubuntu:~/DevOps$
```

Summary:

- You can revisit past versions by checking out a specific commit hash.
- In detached HEAD state, changes are not attached to any branch.
- Always return to your main branch after inspecting old commits.

Task 6 – Resetting Commits (Soft vs Hard Reset) (With Verification Steps)

1. **Add a new line in any file and commit it:**

```
zahra225@ubuntu:~/DevOps$ cd ~/DevOps/Labs/Lab03
zahra225@ubuntu:~/DevOps/Labs/Lab03$ echo "Test line 1" >> temp.txt
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git add .
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git commit -m "Added test line"
[main 97b415a] Added test line
 1 file changed, 1 insertion(+)
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

2. **Add another change and commit again:**

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ echo "Test line 2" >> temp.txt
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git add .
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git commit -m "Second test commit"
[main 7a4c4ee] Second test commit
 1 file changed, 1 insertion(+)
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

3. **View the history before reset:**

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git log --oneline -4
7a4c4ee (HEAD -> main) Second test commit
97b415a Added test line
e1da49f (origin/main, origin/HEAD) Removed tracked textfiles directory
e4d9ff5 Added .gitignore to ignore textfiles directory
```

4. **Check file contents for both changes:**

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ cat temp.txt
Test line 1
Test line 2
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

5. **Perform a soft reset (keeps changes in working directory):**

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git reset --soft HEAD~1
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

6. **Check commit history after soft reset:**

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git log --oneline -4
97b415a (HEAD -> main) Added test line
e1da49f (origin/main, origin/HEAD) Removed tracked textfiles directory
e4d9ff5 Added .gitignore to ignore textfiles directory
066b8aa Added textfiles directory with three files
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

7. **Verify changes in the file:**

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ cat temp.txt
Test line 1
Test line 2
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

8. **Check git status:**

9. **Perform commit**

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ cat temp.txt
Test line 1
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

10. **Perform a hard reset (discards changes completely):**

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   temp.txt

zahra225@ubuntu:~/DevOps/Labs/Lab03$ git commit -m "Second Test commit"
[main dcaad7d] Second Test commit
 1 file changed, 1 insertion(+)
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git log --oneline -3
dcaad7d (HEAD -> main) Second Test commit
97b415a Added test line
e1da49f (origin/main, origin/HEAD) Removed tracked textfiles directory
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git reset --hard HEAD~1
HEAD is now at 97b415a Added test line
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

11. **Check commit history after hard reset:**

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git log --oneline -4
97b415a (HEAD -> main) Added test line
e1da49f (origin/main, origin/HEAD) Removed tracked textfiles directory
e4d9ff5 Added .gitignore to ignore textfiles directory
066b8aa Added textfiles directory with three files
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

12. **Verify changes in the file:**

13. **Check git status:**

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ cat temp.txt
Test line 1
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

Task 7 – Amending the Last Commit

1. Make a small change in any file.

2. Stage it and commit:

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ cd ~/DevOps
zahra225@ubuntu:~/DevOps$ git checkout main
Already on 'main'
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)
zahra225@ubuntu:~/DevOps$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
zahra225@ubuntu:~/DevOps$ git log --oneline -3
97b415a (HEAD -> main) Added test line
e1da49f (origin/main, origin/HEAD) Removed tracked textfiles directory
e4d9ff5 Added .gitignore to ignore textfiles directory
zahra225@ubuntu:~/DevOps$ cd ~/DevOps/Labs/Lab03
zahra225@ubuntu:~/DevOps/Labs/Lab03$ echo "Log message fix" >> temp.txt
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git add .
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git commit -m "Fix log message"
[main 318a256] Fix log message
 1 file changed, 1 insertion(+)
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git log --oneline -3
318a256 (HEAD -> main) Fix log message
97b415a Added test line
e1da49f (origin/main, origin/HEAD) Removed tracked textfiles directory
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git show --stat HEAD
commit 318a2563dcc2d958a0d51a4a98e3a830cfab7d8d (HEAD -> main)
Author: Zahra Bashir <zahra3214567@gmail.com>
Date:   Sun Oct 11 12:27:20 2026 +0000

    Fix log message

 Labs/Lab03/temp.txt | 1 +
 1 file changed, 1 insertion(+)
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

3. Realize you forgot to change another file. Update it now.

4. Add it and amend the last commit:

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ echo "# compose placeholder" >> compose.yaml
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git status
On branch main
Your branch is ahead of 'origin/main' by 2 commits.
  (use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   compose.yaml

no changes added to commit (use "git add" and/or "git commit -a")


[main 3eee63a] Fix log message and update compose file
 Date: Sun Oct 11 12:27:20 2026 +0000
 2 files changed, 2 insertions(+)
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git log --oneline -3
3eee63a (HEAD -> main) Fix log message and update compose file
97b415a Added test line
e1da49f (origin/main, origin/HEAD) Removed tracked textfiles directory
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git show --stat HEAD
commit 3eee63a3621c591b9d8144118a283d46853f6758 (HEAD -> main)
Author: Zahra Bashir <zahra3214567@gmail.com>
Date:   Sun Oct 11 12:27:20 2026 +0000

    Fix log message and update compose file

 Labs/Lab03/compose.yaml | 1 +
 Labs/Lab03/temp.txt     | 1 +
 2 files changed, 2 insertions(+)
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

Summary:

- git commit --amend lets you modify your last commit, including its message or staged files.
- Useful for quick fixes before pushing to remote.
- Once pushed, amending requires force-push to update remote history.

Task 8 – Reverting a Commit (Safe Undo on Remote Branch)

1. Make a change and commit it:

```
zahra225@ubuntu:~/DevOps$ cd ~/DevOps/Labs/Lab03
zahra225@ubuntu:~/DevOps/Labs/Lab03$ echo "temporary text" >> temp.txt
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git add .
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git commit -m "Added temporary text file"
[main 883fd87] Added temporary text file
 1 file changed, 1 insertion(+)
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git push origin main
Enumerating objects: 21, done.
Counting objects: 100% (20/20), done.
Delta compression using up to 2 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (16/16), 1.37 KiB | 16.00 KiB/s, done.
Total 16 (delta 5), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (5/5), completed with 2 local objects.
To github.com:zahra225/DevOps.git
   e1da49f..883fd87  main -> main
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git log --oneline -3
883fd87 (HEAD -> main, origin/main, origin/HEAD) Added temporary text file
3eee63a Fix log message and update compose file
97b415a Added test line
zahra225@ubuntu:~/DevOps/Labs/Lab03$ cat temp.txt
Test line 1
Log message fix
temporary text
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

2. Now revert that commit safely (do not delete it):

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git revert 883fd87
[main d834534] Revert "Added temporary text file"
 1 file changed, 1 deletion(-)
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git log --oneline -4
d834534 (HEAD -> main) Revert "Added temporary text file"
17e1c7f Revert "Removed tracked textfiles directory"
883fd87 (origin/main, origin/HEAD) Added temporary text file
3eee63a Fix log message and update compose file
zahra225@ubuntu:~/DevOps/Labs/Lab03$ cat temp.txt
Test line 1
Log message fix
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git show --stat HEAD
commit d8345344bacab9b6f1a8b3ae927cf55e6e48509e (HEAD -> main)
Author: Zahra Bashir <zahra3214567@gmail.com>
Date:   Sun Oct 11 12:37:51 2026 +0000

    Revert "Added temporary text file"

    This reverts commit 883fd877d8cdc3b744854563f98517bc37e9f8a.

 Labs/Lab03/temp.txt | 1 -
 1 file changed, 1 deletion(-)
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

3. Push the revert commit:

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git push origin main
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 2 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (11/11), 1007 bytes | 16.00 KiB/s, done.
Total 11 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To github.com:zahra225/DevOps.git
   883fd87..d834534  main -> main
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

Summary:

- git revert <hash> creates a new commit that undoes changes from a specific commit.
- It does not rewrite history, making it safe for shared branches.
- Always use revert (not reset) for undoing changes on remote branches.

Task 9 – Force Push (With Caution)

1. Create a new branch:

```
zahra225@ubuntu:~/DevOps$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
zahra225@ubuntu:~/DevOps$ git checkout -b test-force
Switched to a new branch 'test-force'
zahra225@ubuntu:~/DevOps$ git branch
  feature-branch
  main
* test-force
zahra225@ubuntu:~/DevOps$
```

2. Make and commit a small change.

```
zahra225@ubuntu:~/DevOps$ cd ~/DevOps/Labs/Lab03
zahra225@ubuntu:~/DevOps/Labs/Lab03$ echo "Force push test line" >> temp.txt
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git add temp.txt
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git commit -m "Force push test commit"
[test-force 11ffe0d] Force push test commit
 1 file changed, 1 insertion(+)
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git log --oneline -3
11ffe0d (HEAD -> test-force) Force push test commit
d834534 (origin/main, origin/HEAD, main) Revert "Added temporary text file"
17e1c7f Revert "Removed tracked textfiles directory"
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

3. Push it:

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git push -u origin test-force
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 508 bytes | 42.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'test-force' on GitHub by visiting:
remote:      https://github.com/zahra225/DevOps/pull/new/test-force
remote:
To github.com:zahra225/DevOps.git
 * [new branch]      test-force -> test-force
branch 'test-force' set up to track 'origin/test-force'.
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

4. Perform a hard reset:

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git reset --hard HEAD~1
HEAD is now at d834534 Revert "Added temporary text file"
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git log --oneline -3
d834534 (HEAD -> test-force, origin/main, origin/HEAD, main) Revert "Added tempora
ry text file"
17e1c7f Revert "Removed tracked textfiles directory"
883fd87 Added temporary text file
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git status
On branch test-force
Your branch is behind 'origin/test-force' by 1 commit, and can be fast-forwarded.
  (use "git pull" to update your local branch)

nothing to commit, working tree clean
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

5. Push again

```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git push origin test-force
To github.com:zahra225/DevOps.git
 ! [rejected]        test-force -> test-force (non-fast-forward)
error: failed to push some refs to 'github.com:zahra225/DevOps.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. If you want to integrate the remote changes,
hint: use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

6. Now force-push to remote:

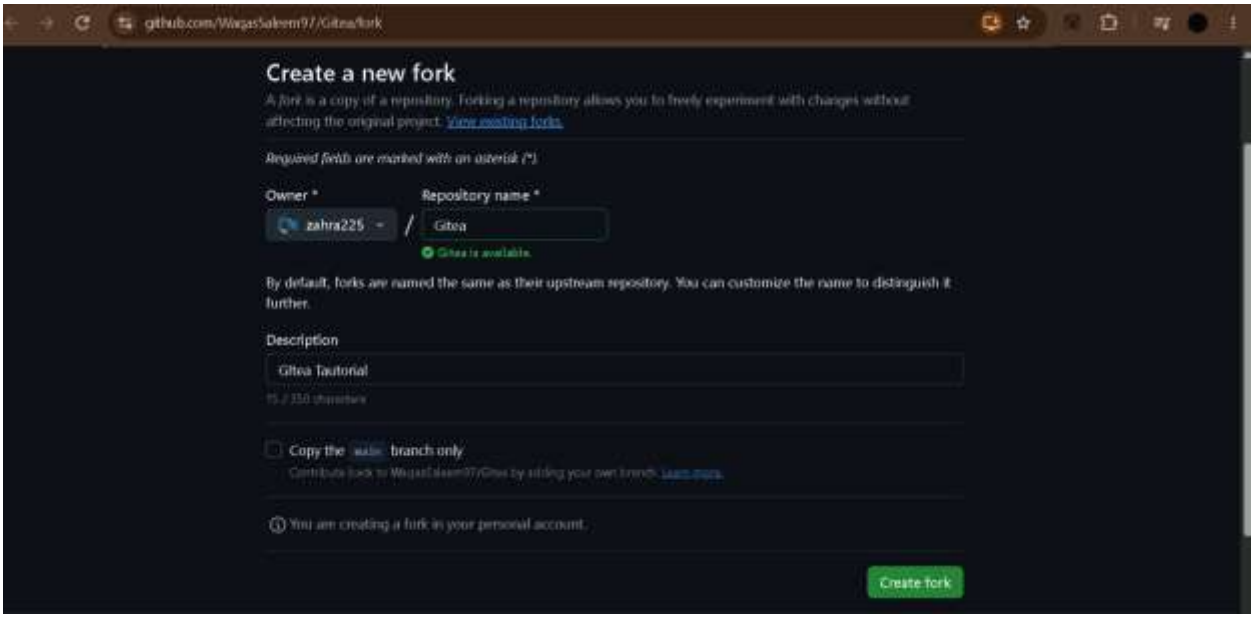
```
zahra225@ubuntu:~/DevOps/Labs/Lab03$ git push origin test-force --force
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:zahra225/DevOps.git
 + 11ffe0d...d834534 test-force -> test-force (forced update)
zahra225@ubuntu:~/DevOps/Labs/Lab03$
```

Summary:

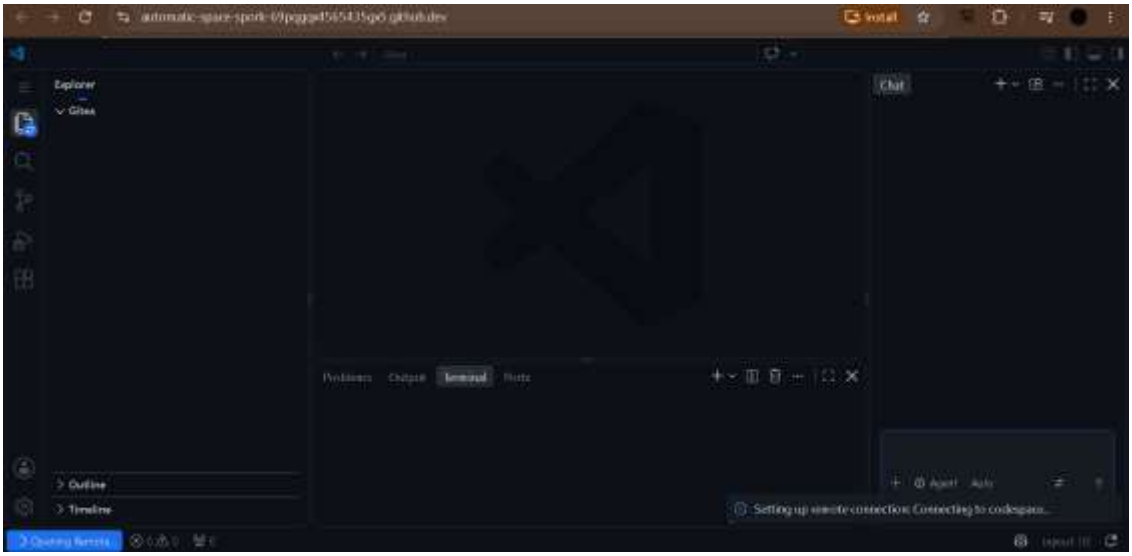
- Force push (git push --force) rewrites remote branch history, overwriting changes.
- Only use on non-shared branches to avoid disrupting collaborators.
- Always double-check before force-pushing.

Task 10 – Running Gitea in GitHub Codespaces via Docker Compose

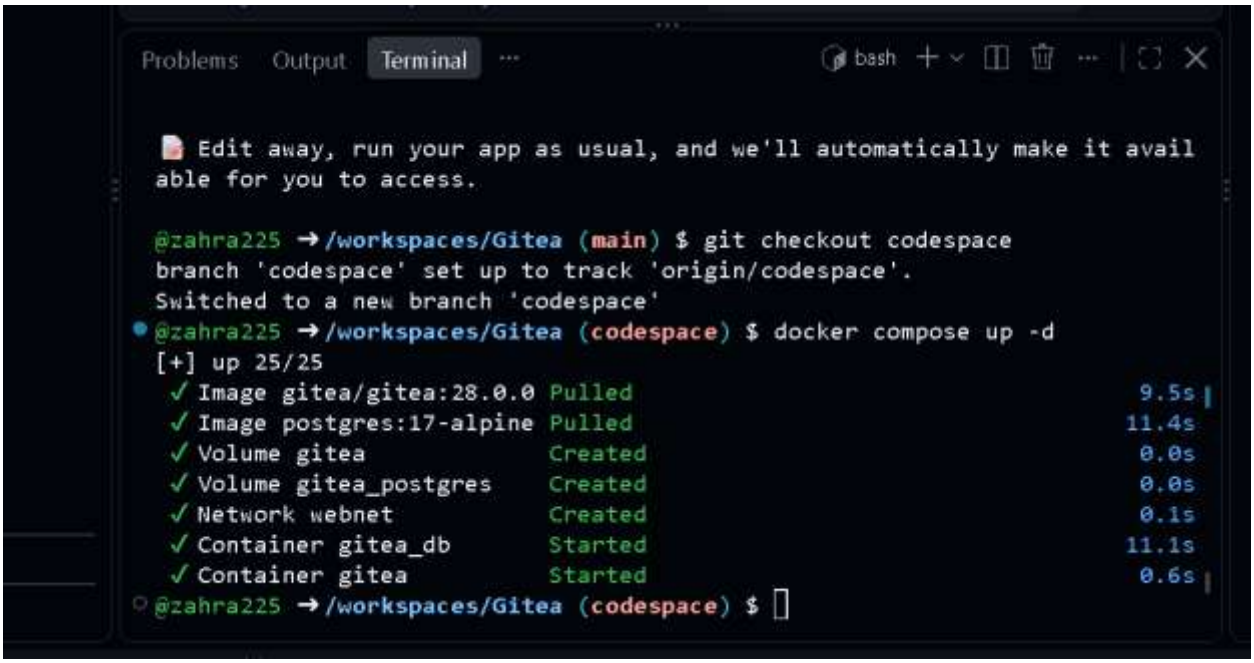
1. **Fork the Gitea Repository**



2. **Open the Forked Repo in GitHub Codespaces**



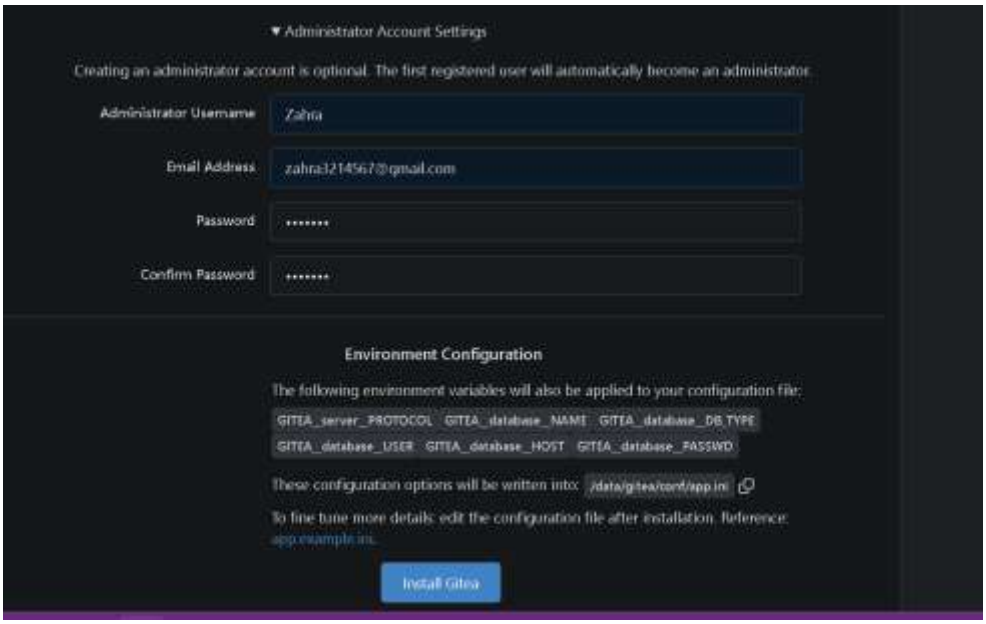
3. **Switch to the codespace Branch and Start Gitea**



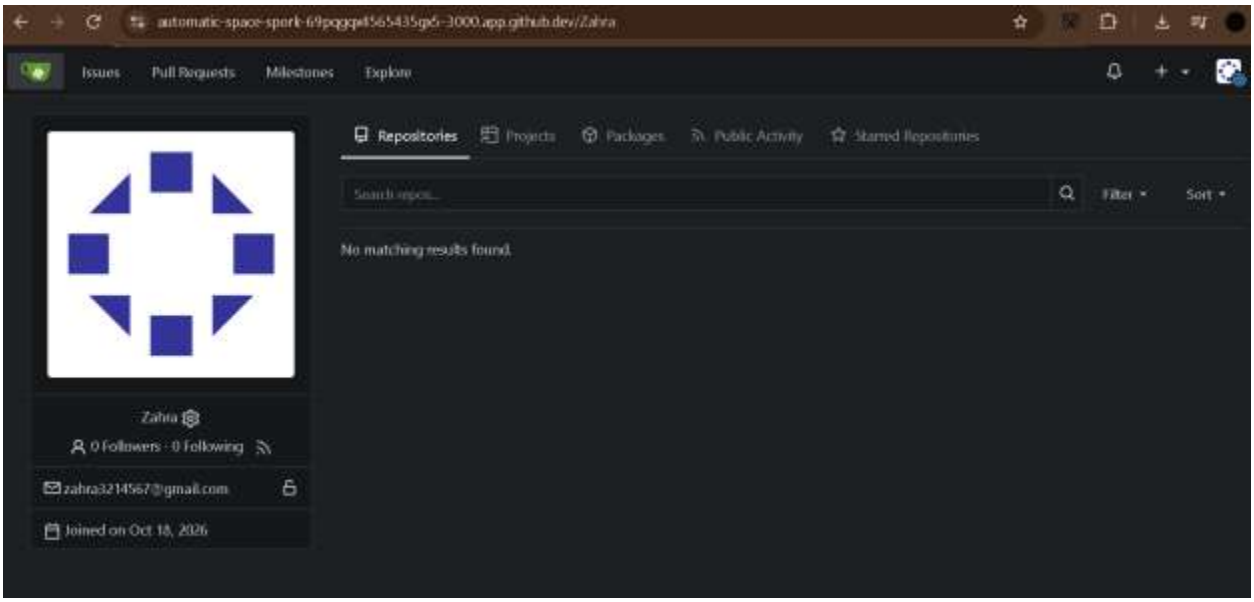
4. **Access Gitea Web Interface**



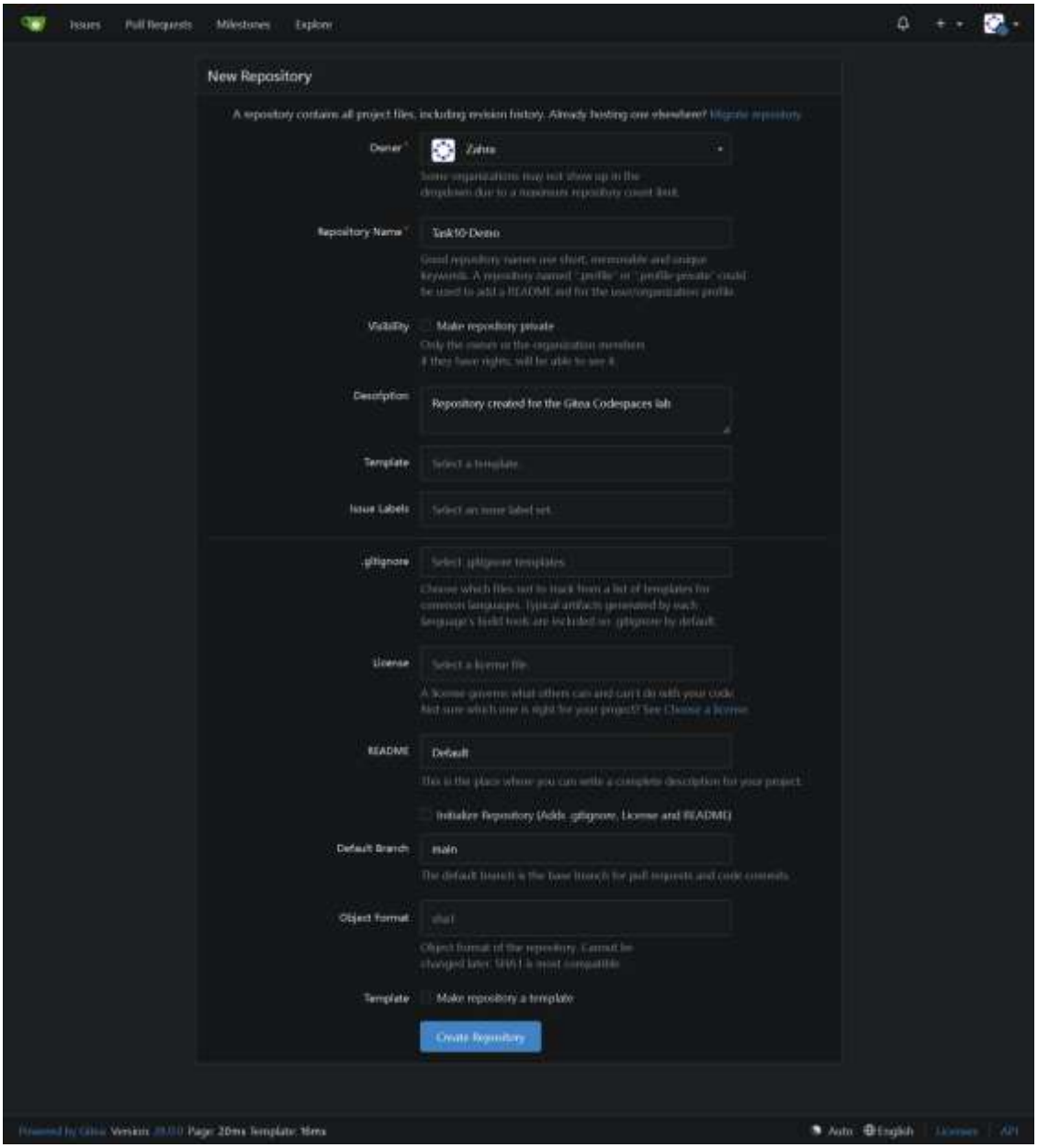
5. **Install Gitea**



6. **Log In to Gitea**

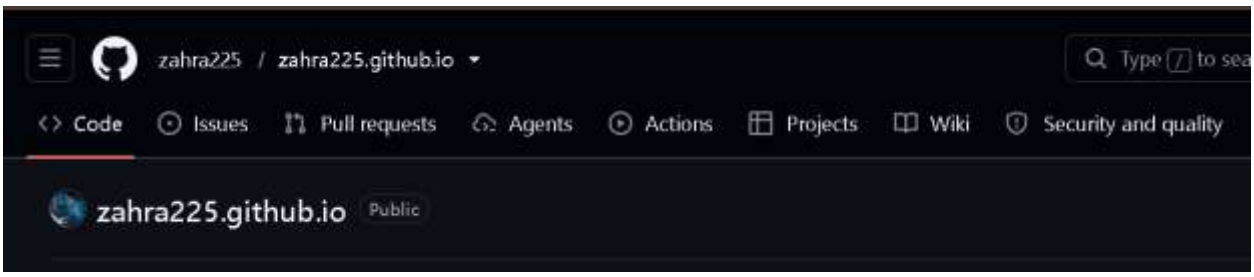


7. **Create a New Repository in Gitea**

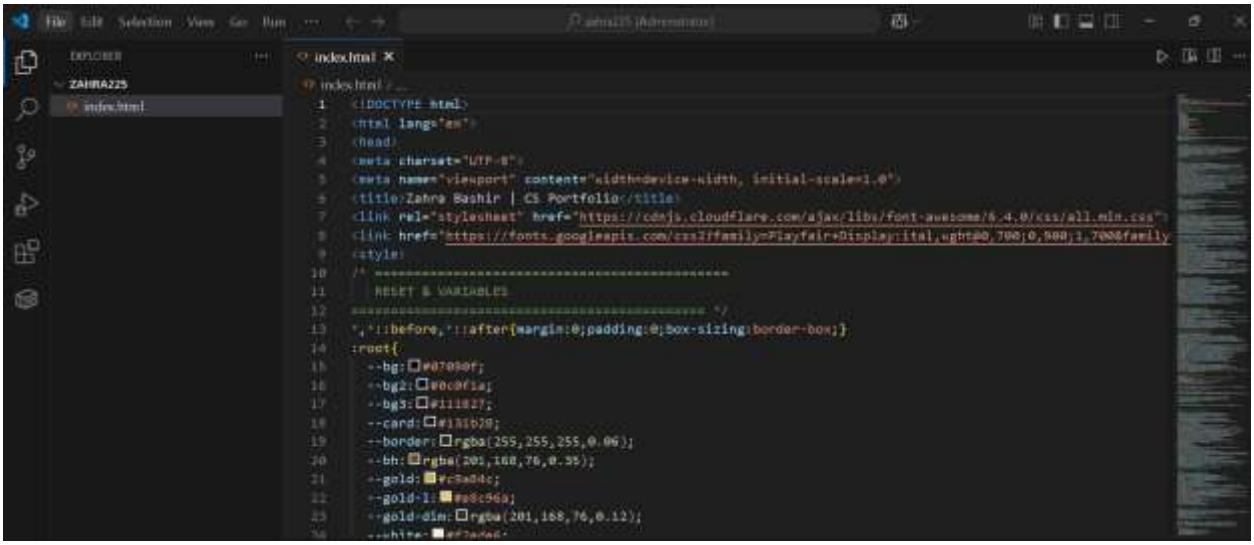


Task 11 – Creating a GitHub Pages Portfolio Site

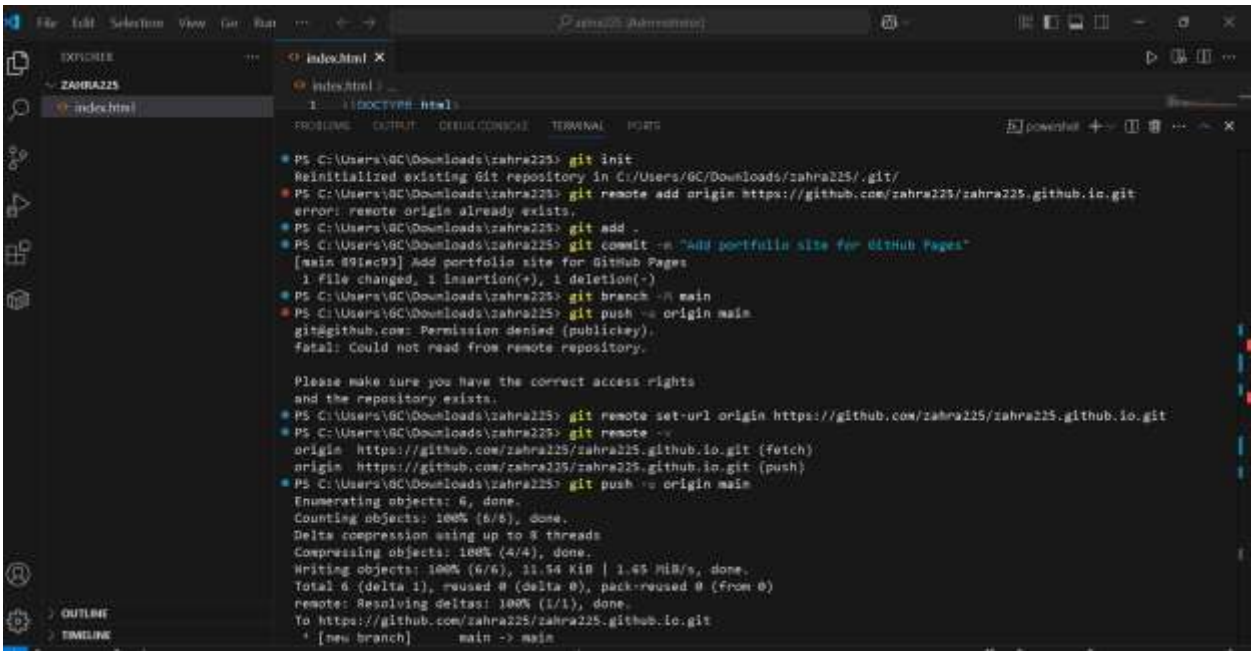
1. **Create a GitHub Pages Repository**



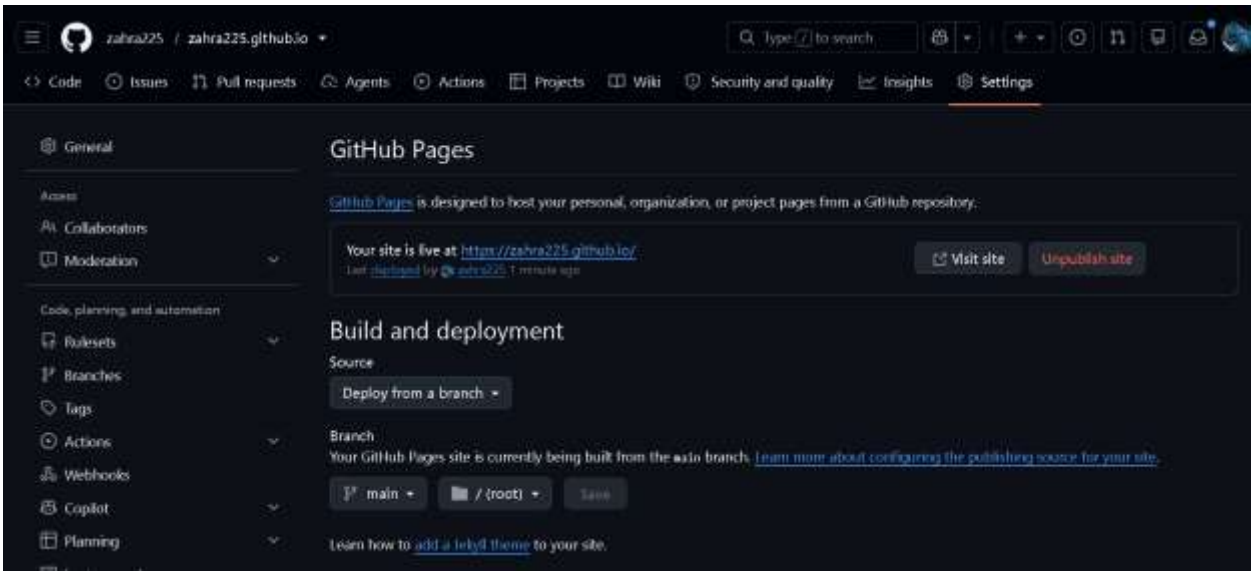
2. **Add Static Website Code**



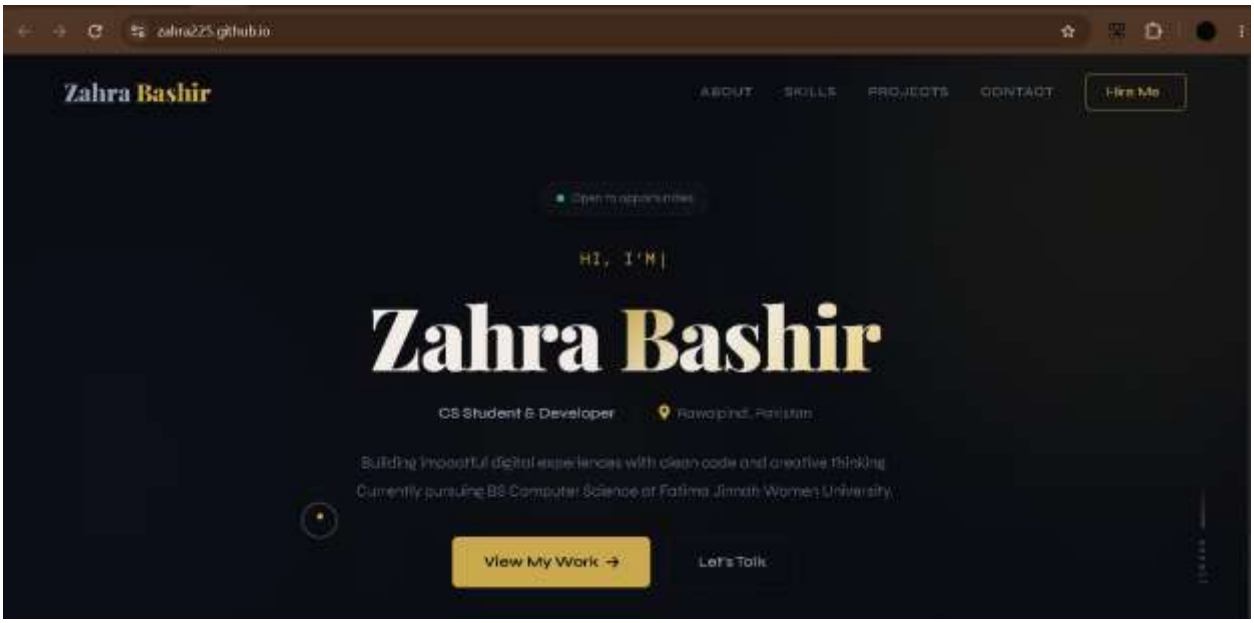
3. **Push the Files to GitHub**



4. **Check GitHub Pages Settings**



5. **Visit Your Live Site**



Live portfolio link: **https://zahra225.github.io/**